

Week 15: Graded Mini Project

This **Week 15 Graded Mini Project** asks you to design and implement a **domain-specific Retrieval-Augmented Generation (RAG)** conversational assistant using **publicly available documents**. Your assistant must retrieve relevant content from your document set and generate answers that stay grounded in that retrieved context. You may use the *RAG-based HR Support Chatbot* discussed in class as a reference architecture, but your submission must be implemented for a **non-HR domain**.

Where to find the tasks

The complete brief, any starter structure (if provided), and the submission environment will be available under '**Week 15 Graded Mini Project**' in [Vocareum](#). Follow the instructions there to implement your solution, run sample tests, and package your submission.

Project Objective

Your objective is to build a **Domain Support Assistant** that:

- answers user questions **only** using the provided documents,
- supports follow-up questions using conversation history,
- avoids hallucination by refusing to answer when information is missing,
- and demonstrates a correct end-to-end **RAG pipeline**.

Choose ONE Domain

Select **one** domain below (or propose your own with publicly available documents):

- Finance – Investment FAQs, mutual fund docs, public filings
- Healthcare – Patient education guides, hospital FAQs
- Education – University handbooks, course catalogues
- E-Commerce – Product manuals, return policies
- Automotive – Vehicle manuals, maintenance guides
- Real Estate – Rental agreements, buyer guides
- Sustainability – ESG reports, climate guidelines
- Legal – Public laws, compliance frameworks
- Technology – Open-source documentation, API manuals

Data Requirements

- **Minimum 3–5 documents**
- Documents must be **publicly available**
- **Allowed formats:** PDF, TXT
- **Possible sources:** Kaggle datasets, official documentation websites, government/public portals

Functional Requirements

1) Document Ingestion

- Load documents from a local folder
- Split content into semantic chunks
- Generate embeddings using **OpenAI embeddings**
- Store embeddings in a vector store (e.g., **FAISS** or similar)

2) Retrieval-Augmented Chat

- Retrieve top-k relevant chunks per query
- Inject retrieved context into the LLM prompt
- Answer **strictly** based on retrieved content

3) Context Awareness

- Maintain conversation history
- Support follow-up questions (e.g., “What about eligibility?” / “Does this apply to part-time users?”)

4) Safety & Accuracy

- If the answer is not in the documents, respond with:
“**I don’t have enough information in the provided documents.**”

Non-Functional Requirements

- Use **OpenAI API**
- Use **LangChain**
- Write modular, readable Python code
- Use environment variables for API keys (**no hard-coded secrets**)

Deliverables

Mandatory

- **ingest.py** – document ingestion and indexing
- **chatbot.py** or **app.py** – RAG chatbot implementation
- **README.md** – project explanation, setup, and run instructions
- **Sample conversation** screenshots or logs

Optional (Bonus)

- Streamlit UI
- Source document citation in answers
- Feedback mechanism

Constraints

- **Do not** use fine-tuning
- **Do not** hard-code answers
- **Do not** rely solely on LLM knowledge
- Retrieval must **influence** responses

Example Interactions (Expected)

- **User:** What is the return policy?
Bot: Based on the return policy document, ...
- **User:** Does this apply to discounted items?
Bot: Yes/No, according to the relevant section in the document, ...

Practice Guidelines (for Graded Mini Project):

This graded mini project will be completed in **Vocareum**. You will build a **RAG-based conversational assistant** using **LangChain** and the **OpenAI API**, focusing on document ingestion, retrieval quality, grounded prompting, and follow-up handling.

To Use Chat Support for This Assignment (inside Vocareum):

- Open your practice notebook in **Vocareum**. Use the left sidebar to open the **Assistant** panel.
- Ask for clarifications, debugging help, or code snippets directly in the **Assistant** chat while you implement your RAG pipeline.

- Monitor your **GenAI Allowance/Budget**, as shown in the **Assistant** panel, and plan prompts accordingly.
- If you exhaust your allowance for a task, please contact the Emeritus support team.
- **API key access:** Your course-specific API key and usage notes are provided via the guide and Vocareum instructions. Follow those steps to locate your key when needed.
- You can find detailed instructions here: [Vocareum GenAI API Students Guide.pdf](#)Download [Vocareum_GenAI_API_Students_Guide.pdf](#)

Security warning: Keep your API key *private*. Do not paste it in screenshots, chats, notebooks you share, or public repos. Prefer environment variables or provided key-management cells; never hard-code keys in source files. If you suspect exposure, rotate the key immediately per the guide.

To access Vocareum:

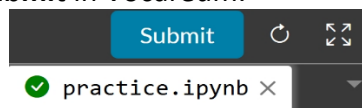
- Go to the *Getting Started* module on your Course homepage (just below Programme Orientation).
- Click the link labelled *Vocareum: Professional Certificate Programme in Agentic AI and Applications*.
- You can also access it from the Course menu on the left side of your screen.

Reminder: Run and test your solution inside **Vocareum** before submission. Ensure ingestion completes successfully, your chatbot can answer and refuse appropriately, and your sample runs/logs are clearly shown.

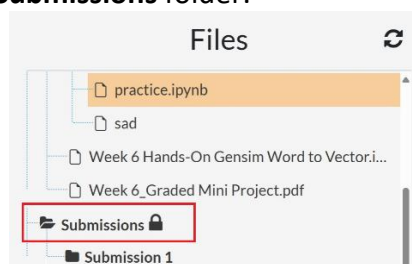
Submission Guidelines

You must submit three components in one zip:

- **Jupyter Notebook:** Include complete code, outputs, and short explanations for all sections (A–D). After completing the project in Vocareum:
 1. Click on **Submit** in Vocareum.



2. Go to the **Submissions** folder.



3. Right-click your notebook file and select **Download** to save it locally.
- **README.md** with:
 - Domain explanation
 - Public data source links
 - Setup & usage instructions
 - Sample conversation screenshots or logs

Name your file as: **Week 15_Graded Mini Project_[Your Name].zip** (or follow the naming convention shown in Vocareum if different).